

Lab 2 – Homework Solutions

September 30, 2024

A Homework

Homework 2.1 (Bisection method). Consider the following equation

$$\cot(x) = \frac{x^2 - 1}{2x}.$$

- Define three intervals containing the three smallest positive roots.
- Find the biggest root among the ones of point 1, using bisection method with tolerance 10^{-6} .
- Plot the evolution of the error with the iterations.

Solution Homework 2.1.

- Define three intervals containing the three smallest positive roots.

```
f = @(x) cot(x);  
g = @(x) (x.^2 - 1) ./ (2*x);  
  
epsilon = 0.01;  
rootfinding_function_plot(f, 0+epsilon, pi-epsilon, true);  
rootfinding_function_plot(f, pi+epsilon, 2*pi-epsilon, false);  
rootfinding_function_plot(f, 2*pi+epsilon, 3*pi-epsilon, false);  
rootfinding_function_plot(g, 0, 3*pi, false);
```

Three intervals for the roots are then [1,2], [3,4] and [6,7].

- Find the biggest root among the ones of point 1, using bisection method with tolerance 10^{-6} .

```
tol = 1e-6;  
h = @(x) cot(x) - (x.^2 - 1) ./ (2*x);  
  
rootfinding_function_plot(h, 0+epsilon, pi-epsilon, true);  
rootfinding_function_plot(h, pi+epsilon, 2*pi-epsilon, false);  
rootfinding_function_plot(h, 2*pi+epsilon, 3*pi-epsilon, false);  
  
I = [6.5 7] % cannot use [6, 7] since h is discontinuous there  
[xi, x] = bisection(h, I(1), I(2), tol);  
xi  
iter = numel(x)
```

- Plot the evolution of the error with the iterations.

Since the exact value of the roots is **not** known, we perform again the bisection method with a very small tolerance, and consider the final approximation as the exact root of the equation.

figure

```
% First root
[xiex, xex] = bisection(h, I(1), I(2), 1e-10*tol);
xiex
err = abs(x - xiex);

subplot(1,2,1)
semilogy(err, 'LineWidth',2)
box on, hold on
semilogy(tol*ones(iter,1), 'r--', 'LineWidth',2)
xlim([0 iter+1])
set(gca,'FontSize',16)
xlabel('Iteration','FontSize',16)
ylabel('Error','FontSize',16)
```

Homework 2.2 (Newton method). a. Approximate the root x^* of $\tan(x^*) = 2x^*$ in the interval $(0, \frac{\pi}{2})$ up to a tolerance of 10^{-15} .

b. Approximate the root of $\sin(x)$ in the interval $[-\frac{\pi}{2}, \frac{\pi}{2}]$ using Newton method and starting from the value $x^{(0)} = x^*$. What do you expect? What do you observe?

Solution Homework 2.2.

```
%% Homework 2.1
```

a. Approximate the root x^* of $\tan(x^*) = 2x^*$ in the interval $(0, \frac{\pi}{2})$.

```
f = @(x) tan(x) - 2*x;
epsilon = 0.01;
rootfinding_function_plot(f, 0+epsilon, pi/2-epsilon, true);
```

From the plot we choose the interval $I = [1, 1.5]$ that ensures the validity of the Bolzano's theorem, and we can use e.g. the bisection method to find x^* .

```
xstar = bisection(f, 1, 1.5, 1e-15);
```

b. Approximate the root of $\sin(x)$ in the interval $[-\frac{\pi}{2}, \frac{\pi}{2}]$ using Newton method and starting from the value $x^{(0)} = x^*$. What do you expect? What do you observe?

The Newton method for the approximation of $\sin(x) = 0$ is

$$x^{(k+1)} = x^{(k)} - \frac{\sin(x^{(k)})}{\cos(x^{(k)})} = x^{(k)} - \tan(x^{(k)}).$$

Therefore, if $x^{(0)} = x^*$

$$\begin{aligned} x^{(1)} &= x^{(0)} - \tan(x^{(0)}) = x^* - \tan(x^*) = x^* - 2x^* = -x^* = -x^{(0)} \\ x^{(2)} &= x^{(1)} - \tan(x^{(1)}) = -x^* + \tan(x^*) = -x^* + 2x^* = x^* = x^{(0)} \end{aligned}$$

The method is describing the orbit $\{x^*, -x^*\}$, and thus cannot converge!

```
g = @(x) sin(x);
dg = @(x) cos(x);

newton_plot(g, dg, xstar, 1e-6, 25);
```

The first 10 iterations of the Newton method seem to describe the orbit $\{x^*, -x^*\}$, but after the first few iterations some numerical error arises and this makes the algorithm move from the orbit and then to converge to the solution.

Homework 2.3 (Newton method). Consider the following function

$$f(x) = e^{ax} - 1 = 0, \quad a \neq 0.$$

For any value of the parameter a the unique solution is clearly $\xi = 0$.

- Consider the case of $a = 200$ and apply Newton method starting from $x^{(0)} = 1$ with tolerance values of 10^{-3} and 10^{-12} .
- Do the same for $a = 10^{-3}$.
- For each case compare the absolute value of the residual with the error. Motivate the obtained results.

Solution Homework 2.3.

- Consider the case of $a = 200$ and apply Newton method starting from $x^{(0)} = 1$ with tolerance values of 10^{-3} and 10^{-12} .

```
a = 200;
f = @(x) exp(a*x) - 1;
df = @(x) a*exp(a*x);

maxit = 1000;
x0 = 1;

[xi, x] = newton(f, df, x0, 1e-3, maxit);
err_1_3 = abs(xi)
absres_1_3 = abs( f(xi) )

[xi, x] = newton(f, df, x0, 1e-12, maxit);
err_1_12 = abs(xi)
absres_1_12 = abs( f(xi) )
```

- Do the same for $a = 10^{-3}$.

```
a = 1e-3;
f = @(x) exp(a*x) - 1;
df = @(x) a*exp(a*x);

maxit = 1000;
x0 = 1;

[xi, x] = newton(f, df, x0, 1e-3, maxit);
err_2_3 = abs(xi)
absres_2_3 = abs( f(xi) )

[xi, x] = newton(f, df, x0, 1e-12, maxit);
err_2_12 = abs(xi)
absres_2_12 = abs( f(xi) )
```

- For each case compare the absolute value of the residual with the error. Motivate the obtained results.

```
disp('   tol      absres      error')
disp([1e-3 absres_1_3 err_1_3;
      1e-12 absres_1_12 err_1_12])

disp('   tol      absres      error')
disp([1e-3 absres_2_3 err_2_3;
      1e-12 absres_2_12 err_2_12])
```

We know that $|f(x)| < |f'(\xi)| |x - x_i|$. Hence if the derivative of f at x_i is big the residual can be big while the solution is already well approximated. Conversely if the derivative is small, the residual decreases fast and the convergence criterion can result to be too weak.

Newton method requires the analytical expression of $f'(x)$. To avoid this limitation the derivative could be approximated.

Secant method

Using $f'(x^{(k)}) \simeq \frac{f(x^{(k)}) - f(x^{(k-1)})}{x^{(k)} - x^{(k-1)}}$ [lex] $x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})(x^{(k)} - x^{(k-1)})}{f(x^{(k)}) - f(x^{(k-1)})}$ superlinear order $p = \frac{\sqrt{5}+1}{2} \simeq 1.61$.

Steffensen method

Using $f'(x^{(k)}) \simeq \frac{f(x^{(k)} + f(x^{(k)})) - f(x^{(k)})}{f(x^{(k)})}$ [lex] $x^{(k+1)} = x^{(k)} - \frac{(f(x^{(k)}))^2}{f(x^{(k)} + f(x^{(k)})) - f(x^{(k)})}$ second order as Newton method.

Homework 2.4 (Secant method, Steffensen method). Implement both the secant method and Steffensen method. Test them on a problem and compare the convergence rate with respect to Newton method.

Solution Homework 2.4.

```
function [xi, x_iter] = secant(f, x0, x1, tol, maxit)
%SECANT Find a root of the equation f(x) = 0 using the secant method, starting from the
initial guess x0.
%
% [xi, x_iter] = SECANT(f, x0, x1, tol, maxit)
%
% Inputs : f = function handle to the function f(x)
%          x0, x1 = initial guess
%          tol = requested tolerance
%          maxit = maximum number of iterations
% Output :
%          xi = approximation of the root
%          x_iter = vector of the approximations of the root at each step

x_iter(1) = x0;
x_iter(2) = x1;

for (iter = 2:maxit)

    x_iter(iter+1) = x_iter(iter) - f(x_iter(iter)) / ( (f(x_iter(iter)) - f(x_iter(iter-1))) /
(x_iter(iter) - x_iter(iter-1)) );

    if (abs (x_iter(iter+1) - x_iter(iter)) < tol)
        break;
    end

end

xi = x_iter(end);

end
```

File secant.m

```
function [xi, x_iter] = steffensen(f, x0, tol, maxit)
%STEFFENSEN Find a root of the equation f(x) = 0 using the secant method, starting from the
initial guess x0.
%
% [xi, x_iter] = SECANT(f, x0, tol, maxit)
%
% Inputs : f = function handle to the function f(x)
%          x0 = initial guess
%          tol = requested tolerance
%          maxit = maximum number of iterations
% Output :
%          xi = approximation of the root
%          x_iter = vector of the approximations of the root at each step
```

```

x_iter(1) = x0;

for (iter = 1:maxit)

    x_iter(iter+1) = x_iter(iter) - f(x_iter(iter))/ ( (f(x_iter(iter)) + f(x_iter(iter))) - f(
    x_iter(iter))) / f(x_iter(iter)) );

    if (abs (x_iter(iter+1) - x_iter(iter)) < tol)
        break;
    end

end

xi = x_iter(end);

end

```

File steffensen.m

```

edit secant.m
edit steffensen.m

f = @(x) exp(-(x-1).^2) - 1/2;
rootfinding_function_plot(f, -1, 3, true);

% For Newton method
df = @(x) -2*(x-1)*exp(-(x-1).^2);
[xin, xn] = newton(f, df, 2, 1e-4, 1000);
xin
itern = numel(xn)

% 'Exact' solution
[xiex, xex] = newton(f, df, 2, 1e-12, 1000);
errn = abs(xn - xiex);

% Secant
[xisec, xsec] = secant(f, 3, 2, 1e-4, 1000);
xisec
itersec = numel(xsec)
errsec = abs(xsec - xiex);

% Steffensen
[xistef, xstef] = steffensen(f, 2, 1e-4, 1000);
xistef
iterstef = numel(xstef)
errstef = abs(xstef - xiex);

p_n = diff( log(errn(2:end)) ) ./ diff( log(errn(1:end-1)) )
p_sec = diff( log(errsec(2:end)) ) ./ diff( log(errsec(1:end-1)) )
p_stef = diff( log(errstef(2:end)) ) ./ diff( log(errstef(1:end-1)) )

% The errors are the theoretical ones

% Comparison of the errors
figure
semilogy(errn, 'rs-', 'LineWidth', 2)
hold on, box on
semilogy(errsec, 'bs-', 'LineWidth', 2)
semilogy(errstef, 'ks-', 'LineWidth', 2)
set(gca, 'FontSize', 16)
set(gca, 'LineWidth', 1.5)
xlabel('iterations', 'FontSize', 16)
ylabel('error', 'FontSize', 16)
h = legend('Newton', 'Secant', 'Steffensen');
set(h, 'FontSize', 16)

```

Homework 2.5 (Newton method). The equation $x^3 - x^2 - x + 1 = 0$ has a double root in $\xi = 1$. Given $x_0 = 2$, does Newton method converge to ξ ? How can you improve the method for approximating ξ ? Compare the results.

Solution Homework 2.5.

```
f = @(x) x.^3 - x.^2 - x + 1;
df = @(x) 3*x.^2 - 2*x - 1;
rootfinding_function_plot(f, -2, 2, true);

tol = 1e-6;
nmax = 100;
[xin, xn] = newton(f, df, 2, tol, nmax);
xin
itern = numel(xn)

[xinmod, xnmod] = newton(f, df, 2, tol, nmax, 2);
xinmod
iternmod = numel(xnmod)

err_newton = abs(xn - 1);
err_newton2 = abs(xnmod - 1);

figure
semilogy(err_newton, 'bs--', 'LineWidth', 2)
hold on, box on
semilogy(err_newton2, 'ro-', 'LineWidth', 2)
axis([0 itern+1 1e-12 10])
set(gca, 'FontSize', 16)
set(gca, 'LineWidth', 1.5)
xlabel('iterations', 'FontSize', 16)
ylabel('error', 'FontSize', 16)
h = legend('Newton', 'Modified Newton');
set(h, 'FontSize', 16)
```

Homework 2.6 (Bisection method). Consider the following equation

$$e^{-(x-2)^2} = 1 - e^{x-4}.$$

- Plot the function and determine the number of real solutions.
- Apply the bisection method with tolerance 10^{-3} with initial intervals $I_1 = [0, 5]$, $I_2 = [1, 6]$, $I_3 = [1, 5]$ and $I_4 = [-1, 5]$. Can you find all the roots?

Solution Homework 2.6.

- Plot the function and determine the number of real solutions.

```
f = @(x) exp(-(x-2).^2) + exp(x-4) - 1;
rootfinding_function_plot(f, 0, 5);
There are three solutions in the interval [1, 5].
```

- Apply the bisection method with tolerance 10^{-3} with initial intervals $I_1 = [0, 5]$, $I_2 = [1, 6]$, $I_3 = [1, 5]$ and $I_4 = [-1, 5]$. Can you find all the roots?

```
tol = 1e-6;

I1 = [0 5];
[xi1, x_iter1] = bisection(f, I1(1), I1(2), tol);
xi1

I2 = [1 6];
[xi2, x_iter2] = bisection(f, I2(1), I2(2), tol);
xi2

I3 = [1 5];
[xi3, x_iter3] = bisection(f, I3(1), I3(2), tol);
```

xi3

```
I4 = [-1 5];  
[xi4, x_iter4] = bisection(f, I4(1), I4(2), tol);  
xi4
```

In the first and fourth interval the bisection method converges to the root with minimum magnitude; similarly, in the second and third interval the bisection method converges to the root with maximum magnitude. The second root cannot be reached. This is caused by the iterative procedure that halves the interval: at the very first iteration the half-interval that contains the second root is discarded.

```
bisection_plot(f, I2(1), I2(2), tol);  
bisection_plot(f, I3(1), I3(2), tol);
```

This example shows that the choice of the initial interval is important.